轻量的多智能体协作系统 -- Agent 开发必做实战项目

📌 项目定位

主智能体像团队负责人一样，调度三个子智能体（网络搜索、数据库查询、知识库检索）协同完成复杂任务。代码极简但架构完整，是学习多智能体系统的绝佳入口。

🎯 适合找什么方向的工作?

这个项目的知识点覆盖了当前 AI 招聘市场最热的几个方向：

岗位方向	匹配度	为什么
AI Agent 应用工程师	⭐⭐⭐⭐⭐	完整实现了 Orchestrator 多智能体协作模式，这是大 Agent 系统的标配架构
大模型应用开发工程师	⭐⭐⭐⭐⭐	覆盖 RAG 对接、Prompt 编排、工具调用、流式输出心能力
AI 后端开发（工程落地）	⭐⭐⭐⭐	FastAPI + WebSocket 异步架构，证明你具备把 AI 能程化的经验
校招 / 实习（简历项目）	⭐⭐⭐⭐⭐	不到 1000 行代码，没有工作经验的应届生花一周吃透面试能讲清楚每一层设计

👤 适合谁学

人群	理由
正在学 LangChain/LangGraph 的开发者	需要一个完整的、能跑起来的实战项目
对多智能体编排感兴趣的学习者	不想一上来就看 AutoGPT / CrewAI 那种庞然
想理解 FastAPI + WebSocket + Agent 组合的开发者	这是一个真实的全栈 AI 应用
面试前需要 AI Agent 项目撑简历的人	能讲清楚每个设计决策

📚 知识点覆盖

知识点	在项目哪里	面试价值
Orchestrator 多智能体模式	agent/main_agent.py	⭐⭐⭐⭐⭐ 大厂高频考点
LangChain @tool 自定义工具	tools/ 目录	⭐⭐⭐⭐⭐ Agent 岗位标配技能
RAG 知识库对接	tools/ragflow_tools.py	⭐⭐⭐⭐⭐ 招聘高频词
FastAPI 异步 + 后台任务	api/server.py	⭐⭐⭐⭐ 证明你有异步编程实战经验
WebSocket 实时推送	api/monitor.py	⭐⭐⭐⭐ AI 流式输出的标配方案
ContextVar 协程级数据隔离	api/context.py	⭐⭐⭐ 进阶加分项，体现对并发模型的理解
Prompt Engineering 实战	prompt/prompts.yml	⭐⭐⭐ 基础能力，但必须会
Agent 文件操作安全	utils/path_utils.py	⭐⭐⭐ 工程化思维的表现
数据库自然语言查询	tools/db_tools.py	⭐⭐⭐ Text-to-SQL 典型场景

🏗️ 架构速览

代码块

```
用户请求 (POST /api/task)
        |
        ▼
api/server.py ← 入口：FastAPI 路由 + WebSocket
        |
        ▼
agent/main_agent.py ← 核心：主智能体创建、异步流式执行
        ├──→ 子智能体1：网络搜索 (tools/tavily_tool.py)
        ├──→ 子智能体2：数据库查询 (tools/db_tools.py)
        ├──→ 子智能体3：RAGFlow 知识库 (tools/ragflow_tools.py)
        └──→ 主智能体自己：生成 Markdown → 转 PDF
        |
        ▼
(每个步骤通过 WebSocket 实时推送)
api/monitor.py ← 埋点监控 + 事件循环归属判断
        |
        ▼
前端收到实时进度："正在搜索网络..." → "正在查数据库..." → "正在生成文档..."
```

⚡ 5 分钟快速上手

环境要求

- Python 3.10+
- OpenAI 兼容的 LLM API Key（阿里云百炼 / DeepSeek / OpenAI 均可）
- Tavily API Key（有免费额度）

安装步骤

代码块

```
# 1. 克隆项目
git clone https://github.com/waseens/deep-search-pro
cd deep-search-pro

# 2. 创建虚拟环境
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate

# 3. 安装依赖
pip install -r requirements.txt

# 4. 配置环境变量
cp .env.example .env
# 编辑 .env，填入你的 API Key

# 5. 启动服务
python api/server.py
```

验证服务

- 用 Postman 或 curl 调 `POST /api/task`，传一个简单问题，看是否有返回
- 看到 `Uvicorn running` 即表示启动成功

📖 学习建议

按基础分

你的基础	建议时长	学习路径
有 Python + 后端经验	3-5 天	直接跑起来 → 读核心模块 → 改代码验证
Python 刚入门	2-3 周	先补 FastAPI → 再看 LangChain 工具调用 → 最后读源码
只为面试能讲	2-3 天	跑起来 → 背熟架构图 → 盯住 3 个核心文件

推荐阅读顺序

1. `api/server.py` —— 只看路由定义，知道入口在哪
2. `agent/main_agent.py` —— 核心调度逻辑，看懂主 Agent 绑了哪几个工具
3. `tools/` 目录 —— 三个子 Agent 的工具实现，各看一遍
4. `api/monitor.py` —— 知道 WebSocket 进度是怎么推出去的

⚠️ 动手验证（强烈推荐）

光读代码没用，动手改一行，比读十遍都管用：

- 改 `prompt/prompts.yml` 里的 system_prompt，让主 Agent 优先调数据库而不是网络搜索
- 跑一次看看效果，观察行为变化
- 把 Tavily 换成别的搜索 API，看看要改哪些地方

✅ 学习成果检验

Level 1：跑起来了

- [ ] 项目能在本地启动服务
- [ ] 能用 Postman 或 curl 调通 `/api/task` 接口
- [ ] 能看到 WebSocket 推送的实时进度

Level 2：看懂了

- [ ] 能画出架构图，标注每个模块的职责
- [ ] 能说出主 Agent 调了哪三个子 Agent
- [ ] 能解释 `asyncio.create_task` 是干嘛的
- [ ] 能解释 WebSocket 是怎么推进度的

Level 3：面试 Ready

- [ ] 能用 3 分钟讲清楚项目的核心设计
- [ ] 能说出至少 2 个可以改进的地方
- [ ] 能解释为什么用 ContextVar 做数据隔离

🎯 高频面试题

项目相关

Q1：这个项目的核心架构是什么？

> 采用 Orchestrator 多智能体模式——一个主 Agent 负责拆解用户需求，动态调度三个子 Agent（网络搜索、数据库查询、RAG 知识库检索）协同完成任务。整个系统用 FastAPI 提供异步接口，WebSocket 做实时进度推送。

Q2：主 Agent 怎么知道该调哪个子 Agent?

> 通过 `prompt/prompts.yml` 里的 system_prompt 约束行为。Prompt 定义了每个子 Agent 的职责和调用条件，主 Agent 根据用户问题类型自动判断。

Q3：WebSocket 在项目里怎么用的?

> 在 `api/monitor.py` 里埋点监控，每个工具调用步骤都通过 WebSocket 实时推送到前端。用户能看到“正在搜索网络... → 正在查数据库...”的实时进度。

Q4：ContextVar 是干什么的？为什么用它?

> 在 `api/context.py` 里用 ContextVar 做协程级数据隔离。多用户并发时每个请求的数据不串台，是 Python 异步编程里处理请求上下文的标准做法。

Q5：你觉得这个项目有什么可以改进的地方?

> 子 Agent 之间没有直接通信，所有信息只能通过主 Agent 中转。可以改成子 Agent 之间能互相传递中间结果，减少主 Agent 上下文负担。另外可以加缓存机制，避免重复的网络搜索。

通用 AI Agent 面试题

Q6：Orchestrator 模式和 Peer-to-Peer 模式有什么区别?

> Orchestrator 模式有一个中心调度者负责任务分解和结果汇总，适合流程清晰的场景。Peer-to-Peer 模式没有中心节点，Agent 之间直接通信协商，适合开放式协作。Deep Search Pro 用的是前者。

Q7：LangChain 的 @tool 装饰器是干什么的?

> 把普通 Python 函数包装成 Agent 可调用的工具。Agent 看到工具描述和参数定义后决定何时调用、传什么参数。

Q8：RAG 和普通搜索有什么区别?

> 普通搜索返回原始文档；RAG 先把文档向量化存向量库，检索时做语义匹配，再把检索结果作为上下文喂给 LLM 生成答案。

Q9：Agent 操作本地文件时要注意什么安全问题?

> 防止路径遍历攻击——用户通过 `../` 穿越访问系统文件。项目 `utils/path_utils.py` 实现了 12 种路径场景的防护。

💡 面试话术模板

> “我完整跑通过 Deep Search Pro 这个开源项目。它核心是一个多智能体协作系统，采用 Orchestrator 模式——一个主 Agent 负责拆解用户需求，动态调度三个子 Agent 分别做网络搜索、数据库查询和知识库检索。整个系统用 FastAPI 提供异步接口，WebSocket 做实时进度推送。我最

> 关注的是主 Agent 怎么通过 system_prompt 学会判断该调哪个子 Agent，以及工具调用结果怎么汇总成最终回答。通过这个项目，我理解了 Agent 工作流编排和 LangChain 工具调用的基本套路。如果让我改进，我会让子 Agent 之间能直接通信，减少主 Agent 的中转负担。”

📂 核心文件速查表

文件	行数（约）	核心职责
api/server.py	80	FastAPI 路由、请求入口、后台任务创建
agent/main_agent.py	120	主 Agent 创建、流式执行、工具绑定
tools/tavily_tool.py	40	网络搜索工具封装
tools/db_tools.py	60	数据库查询 + 自然语言转 SQL
tools/ragflow_tools.py	50	RAGFlow 知识库检索对接
api/monitor.py	70	WebSocket 埋点监控、进度推送
api/context.py	30	ContextVar 协程级数据隔离
utils/path_utils.py	100	12 种路径安全防护
prompt/prompts.yml	30	System Prompt 配置